



Árvores



Prof. Rafael Mesquita
rgm@cin.ufpe.br

O que veremos nesta aula?

- Motivação
 - ▣ Estruturas hierárquicas x lineares
- Definição e propriedades de árvores
- Árvores binárias
- Percurso em árvores
- Implementação

Motivação

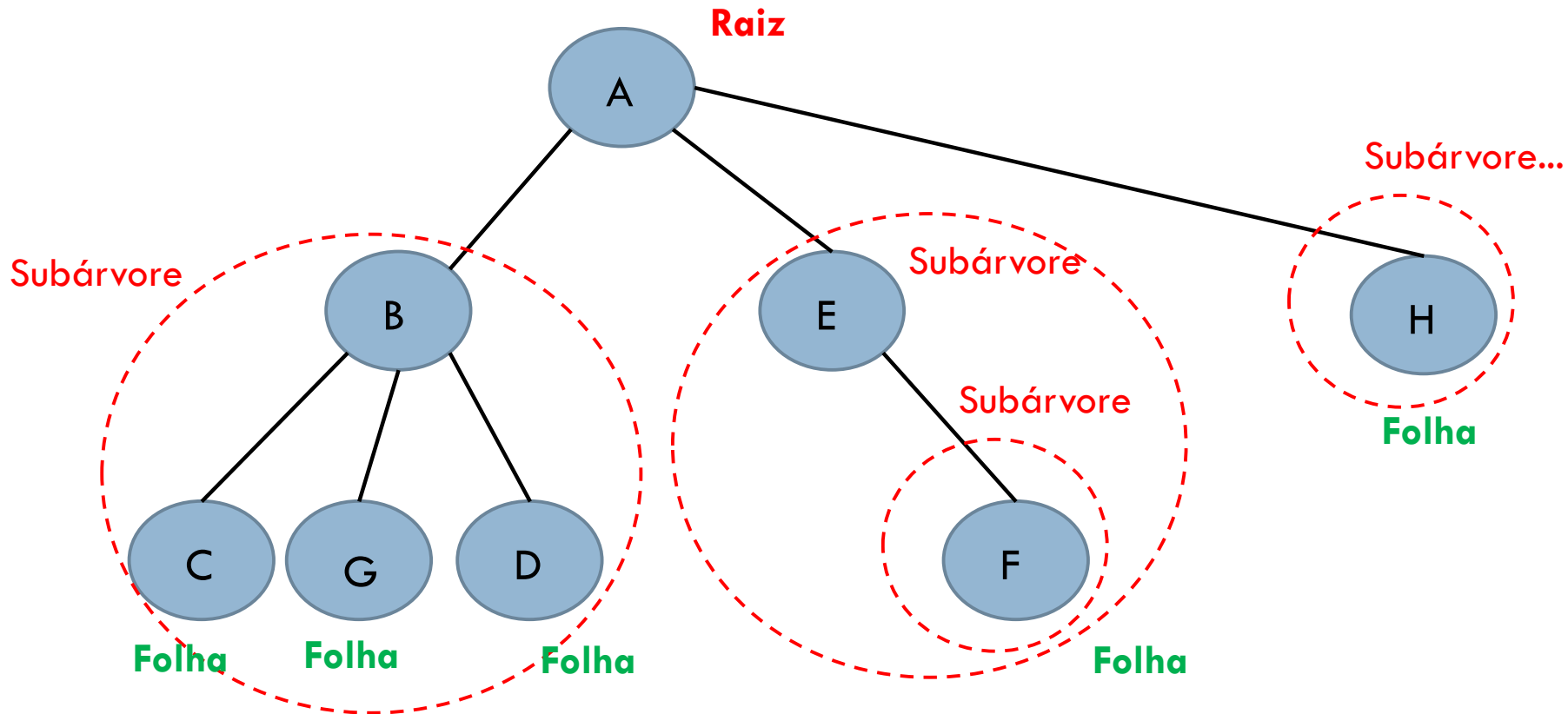
- Vetores e Listas encadeadas representam dados organizados de forma linear
 - ▣ Não são adequados para representar hierarquia
- Exemplo de uso:
 - ▣ sistema de arquivos de um computador

Definição de Árvore

- Composta por um conjunto de nós
- Existe um nó raiz que contém 0 ou mais subárvores
 - ▣ Raízes das subárvores estão diretamente ligadas à raiz
 - ▣ Raízes das subárvores são nós internos
- Cada nó possui um nó pai
 - ▣ Exceto a raiz, que não possui pai
- Um nó que não possui filho é chamado de folha

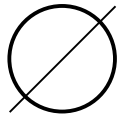
Definição de Árvore

□ Exemplo:



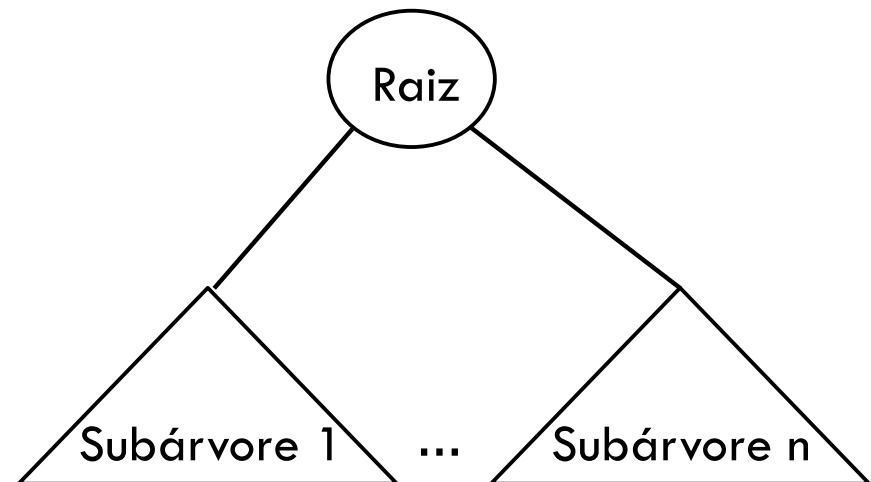
Definição de Árvore

- Cada nó pode possuir zero ou mais filhos
- Árvore pode ser definida recursivamente como:
 - ▣ Uma árvore vazia
 - ▣ Um nó raiz contendo diversas subárvores
 - De acordo com a definição recursiva, qualquer uma das suas subárvores podem ser arvores vazias



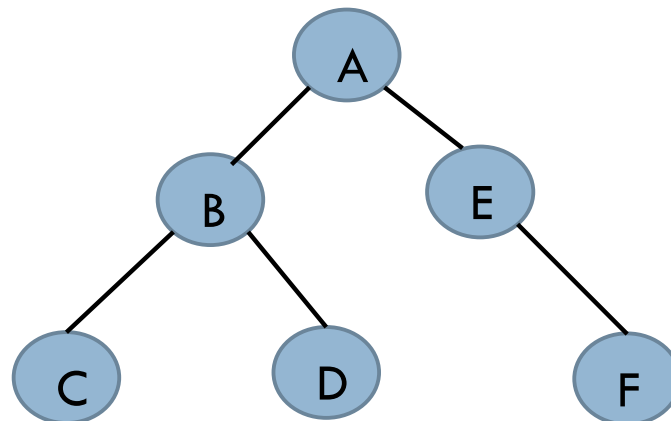
**Árvore
vazia**

OU



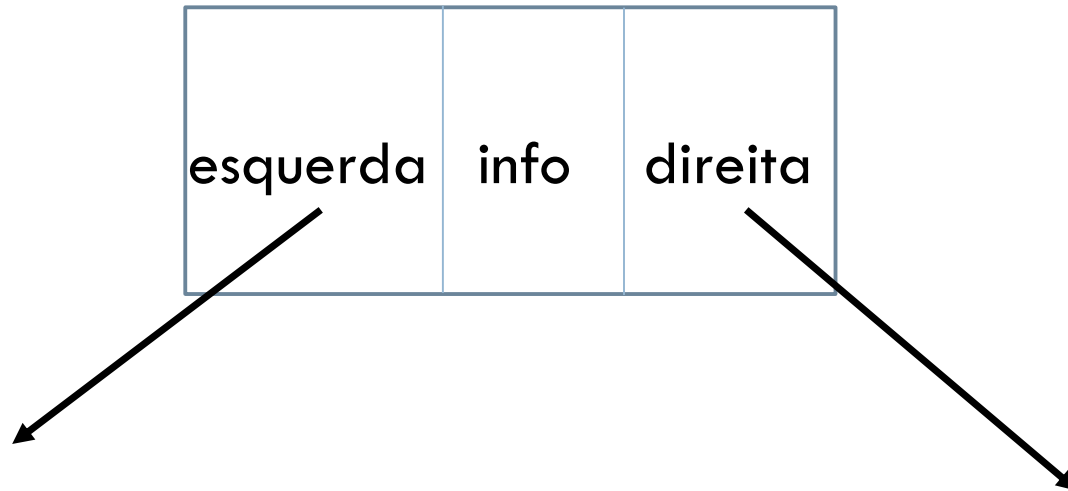
Ávores Binárias

- Tipo específico de árvore em que cada nó pode possuir, no máximo, 2 filhos
 - ▣ Um dos principais tipos de árvores utilizado na literatura e em aplicações
- Foco: árvores binárias!
 - ▣ No entanto, é importante lembrar que **nem toda árvore é uma árvore binária!**



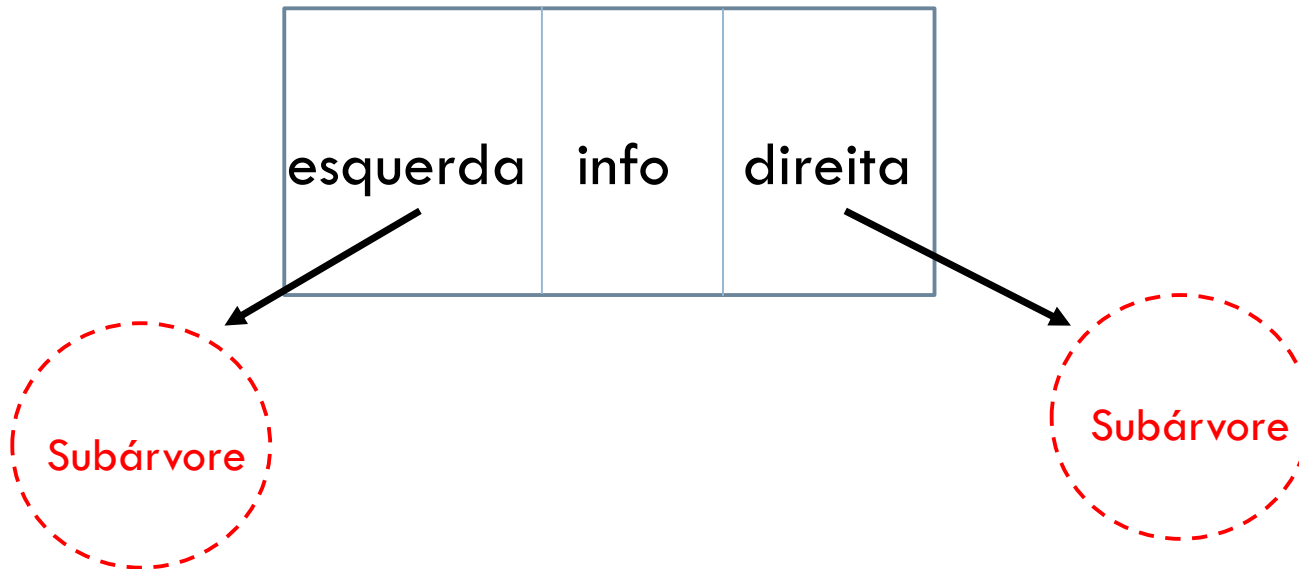
Implementação: árvore binária

```
typedef struct arvore {  
    char info;  
    struct arvore* esquerda;  
    struct arvore* direita;  
}Arvore;
```



Implementação: árvore binária

```
Arvore* cria(char info, Arvore* esquerda, Arvore* direita)
{
    Arvore* nova = (Arvore*)malloc(sizeof(Arvore));
    nova->info = info;
    nova->esquerda = esquerda;
    nova->direita = direita;
    return nova;
}
```



Implementação: árvore binária

```
int vazia(Arvore* raiz)
{
    return raiz == NULL;
}

void libera(Arvore** raiz)
{
    if (!vazia(*raiz))
    {
        libera(&((*raiz)->esquerda));
        libera(&((*raiz)->direita));
        free(*raiz);
        *raiz = NULL;
    }
}
```

Ordens de percurso em árvores binárias

- Em algumas situações é necessário percorrer todos os nós da árvore
- É comum utilizar os seguintes percursos
 - ▣ Pré-ordem
 - Raiz, subárvore esquerda e subárvore direita
 - ▣ Em-ordem (ordem simétrica)
 - Subárvore esquerda, raiz e subárvore direita
 - ▣ Pós-ordem
 - Subárvore esquerda, subárvore direita e raiz

Implementação

```
int main()
{
    Arvore* d = cria('d', NULL, NULL);
    Arvore* c = cria('c', NULL, NULL);
    Arvore* b = cria('b', c, d);
    Arvore* f = cria('f', NULL, NULL);
    Arvore* e = cria('e', NULL, f);
    Arvore* a = cria('a', b, e);
    pre_ordem(a); //a b c d e f
    libera(&(a->direita));
    printf("\n\n");
    pre_ordem(a); //a b c d
    getchar();
    return 0;
}
```

Implementação

□ Pré-ordem

```
void pre_ordem(Arvore* raiz)
{
    if (!vazia(raiz))
    {
        printf("%c ", raiz->info);
        pre_ordem(raiz->esquerda);
        pre_ordem(raiz->direita);
    }
}
```

Implementação

□ Em-ordem

```
void em_ordem(Arvore* raiz)
{
    if (!vazia(raiz))
    {
        em_ordem(raiz->esquerda);
        printf("%c ", raiz->info);
        em_ordem(raiz->direita);
    }
}
```

Implementação

□ Pós-ordem

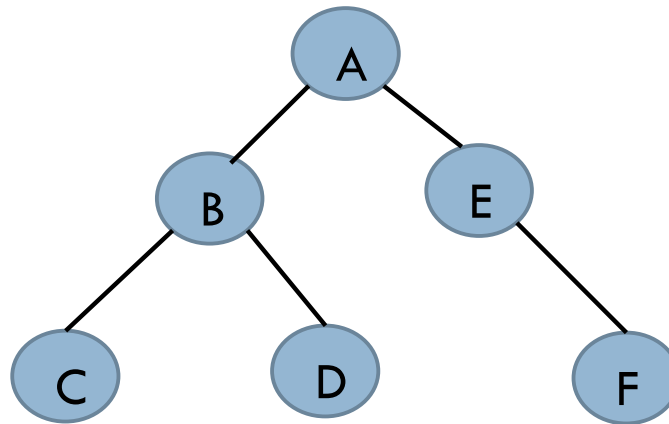
```
void pos_ordem(Arvore* raiz)
{
    if (!vazia(raiz))
    {
        pos_ordem(raiz->esquerda);
        pos_ordem(raiz->direita);
        printf("%c ", raiz->info);
    }
}
```

Ordens de percurso em árvores binárias

- Pré-ordem

- Raiz, subárvore esquerda e subárvore direita

- Ex:



- Saída:

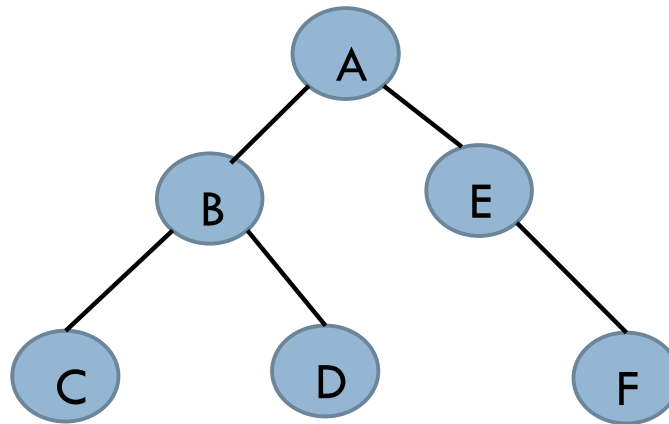
- A,B,C,D,E,F

Ordens de percurso em árvores binárias

- Em-ordem

- subárvore esquerda, raiz e subárvore direita

- Ex:

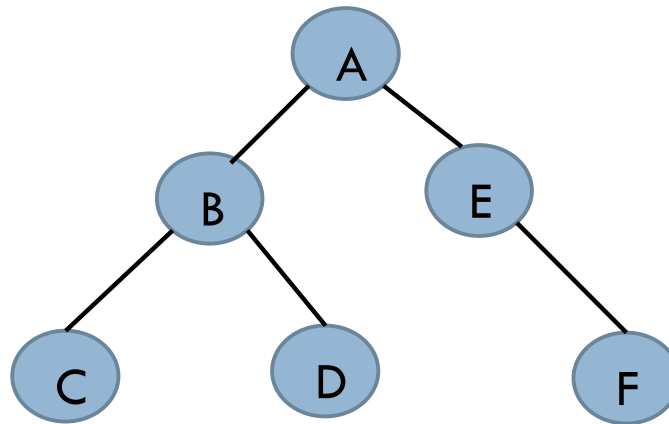


- Saída:

- C,B,D,A,E,F

Ordens de percurso em árvores binárias

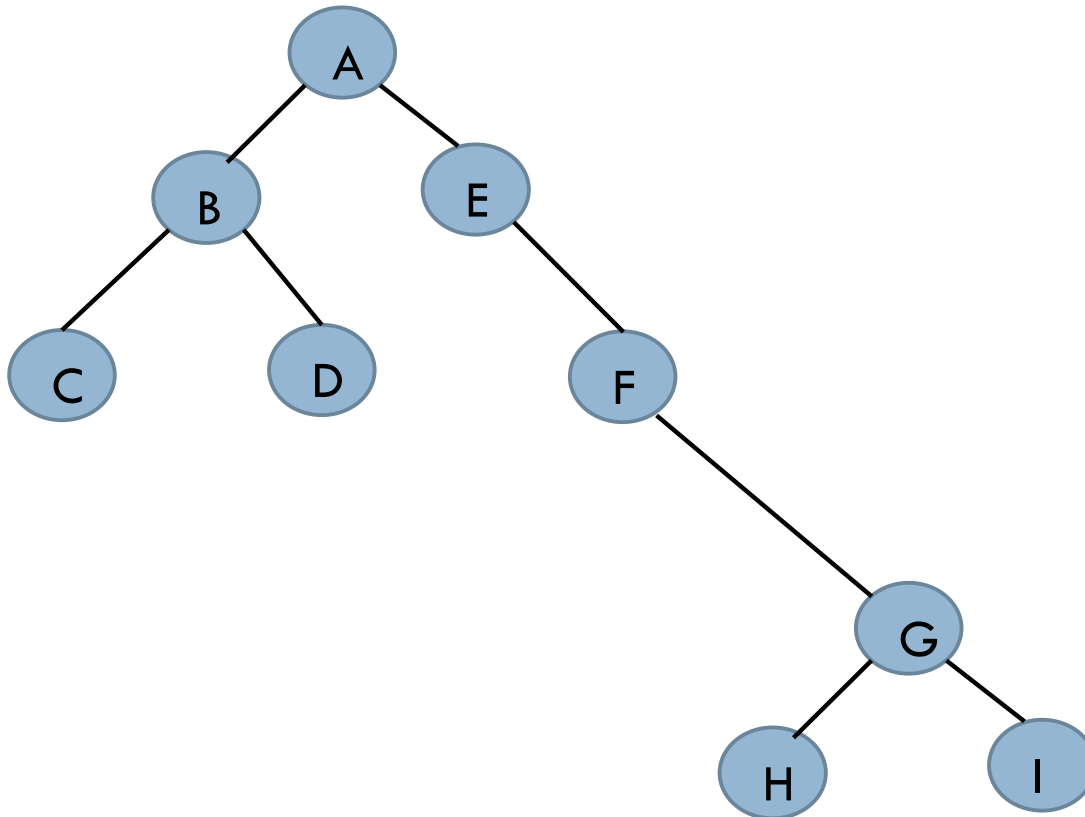
- Pos-ordem
 - subárvore esquerda, subárvore direita e raiz
- Ex:



- Saída:
- C,D,B,F,E,A

Implementação

- Inserindo subárvore em outra árvore



Implementação

□ Inserindo subárvore em outra árvore

```
Arvore* d = cria('d', NULL, NULL);
Arvore* c = cria('c', NULL, NULL);
Arvore* b = cria('b', c, d);
Arvore* f = cria('f', NULL, NULL);
Arvore* e = cria('e', NULL, f);
Arvore* a = cria('a', b, e);
em_ordem(a); // c b d a e f
```

```
Arvore* h = cria('h', NULL, NULL);
Arvore* i = cria('i', NULL, NULL);
Arvore* g = cria('g', h, i);
f->direita = g;
printf("\n");
em_ordem(a); // c b d a e f h g i
getchar();
return 0;
```

Propriedades de Árvores

□ Altura da árvore

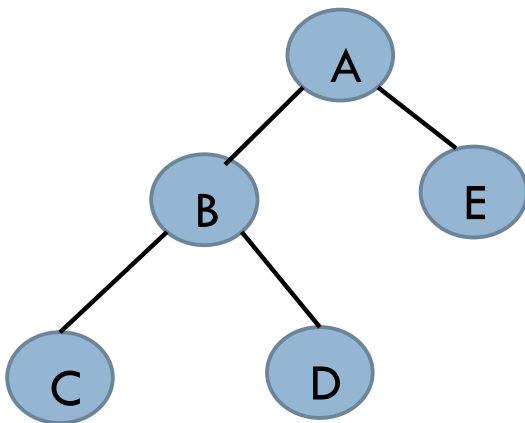
- ▣ Distância do caminho mais longo da raiz até uma das folhas
- ▣ Altura de uma árvore vazia é -1
- ▣ Altura de árvore com um único nó é 0

□ Níveis

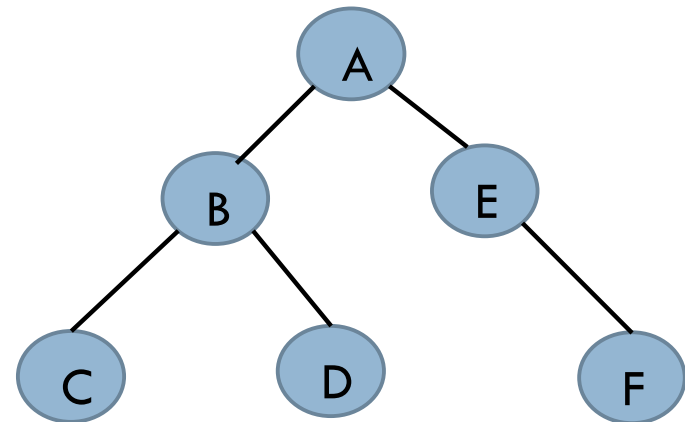
- ▣ Raiz está no nível 0
 - Seus filhos no nível 1, e assim por diante
 - Último nível h é a altura da árvore

Árvore cheia

- Árvore é dita como cheia se todos os seus nós internos possuem todas (duas, no caso de uma árvore binária) as subárvores e todas as folhas estão no último nível



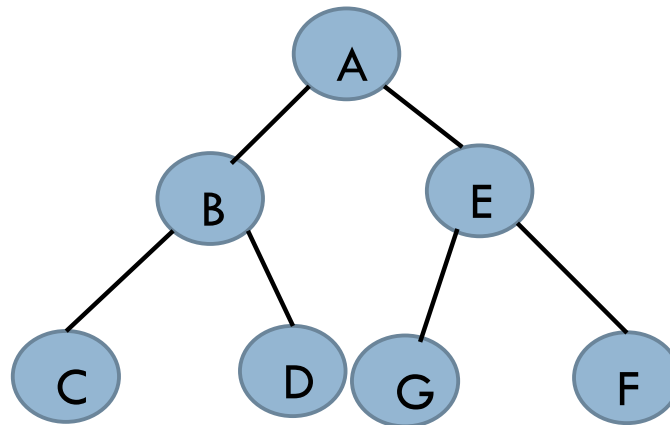
Árvore não cheia: folha fora do último nível!



Árvore não cheia: No interno 'E' não possui todas as subárvores!

Árvore cheia

- Árvore é dita como cheia se todos os seus nós internos possuem todas (duas, no caso de uma árvore binária) as subárvores e todas as folhas estão no último nível

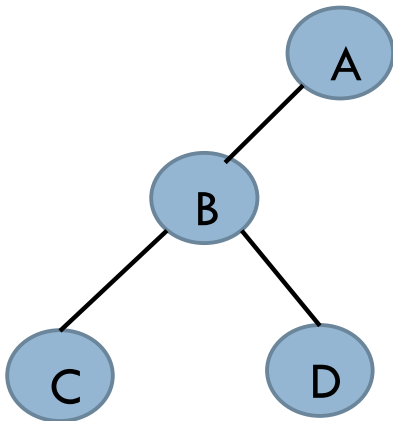


ÁRVORE CHEIA!

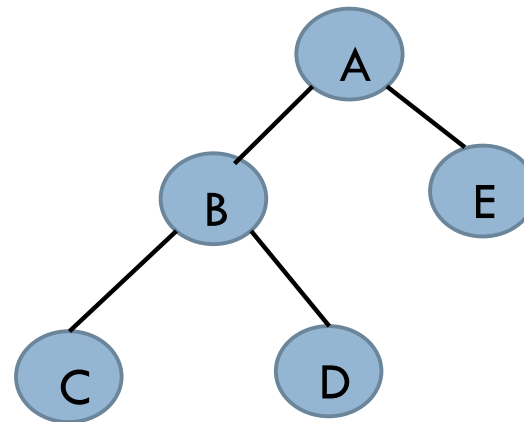
Árvore completa

□ Árvore completa

- ▣ Para uma árvore ser completa, qualquer nó que não possuir uma das subárvores deve estar no último ou penúltimo nível



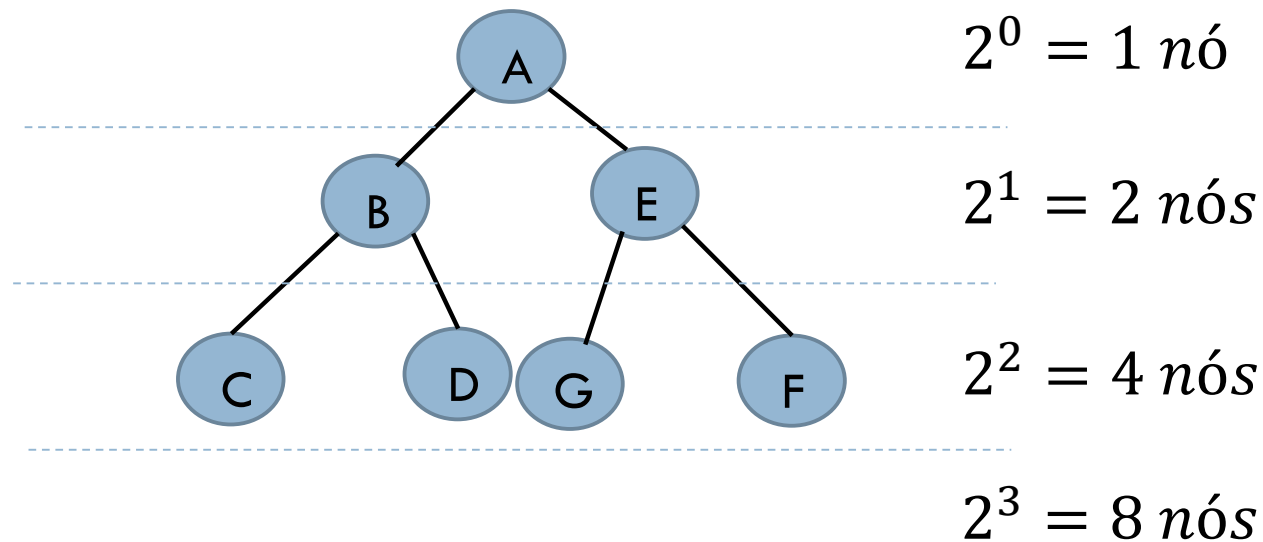
Árvore não completa



Árvore completa

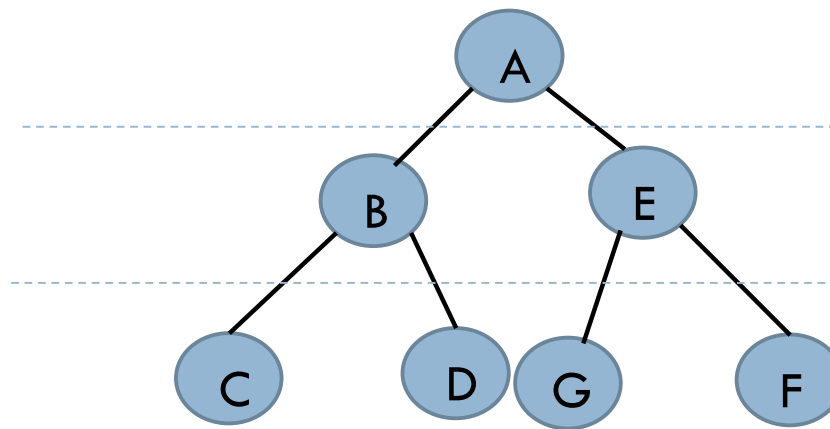
Propriedades de Árvores

□ Número de elementos por nível



Propriedades de Árvores

- Altura X número de nós
 - Árvore binária cheia possui $2^{h+1} - 1$ nós
 - Uma árvore binária completa com n nós possui altura proporcional a $\log_2 n$



Número de nós até o nível

$$2^{0+1} - 1 = 1 \text{ nó}$$

$$2^{1+1} - 1 = 3 \text{ nós}$$

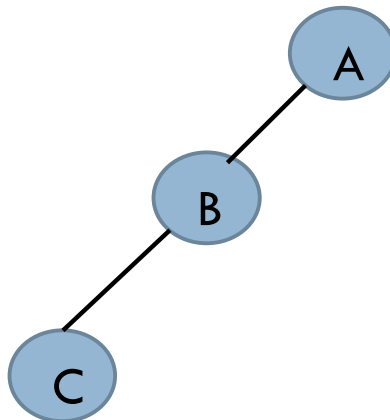
$$2^{2+1} - 1 = 7 \text{ nós}$$

$$2^{3+1} - 1 = 15 \text{ nós}$$

...

Árvore degenerada

- Uma árvore é degenerada se todos os seus nós internos possuem uma única subárvore
- Estrutura hierárquica se degenera em uma estrutura linear
- A altura de uma árvore degenerada de n nós é proporcional a n



Cálculo da altura

- Por que calcular a altura de uma árvore ?
 - ▣ Medida da eficiência no uso da árvore
 - ▣ Reflete o esforço computacional para se percorrer o maior caminho da raiz até uma folha
 - É importante que a altura seja pequena
 - ▣ Ou seja, árvore deve ser o mais próximo possível de uma árvore cheia
 - ▣ Exemplo na aplicação de árvores binárias de busca
 - Quanto maior a altura, maior o esforço computacional máximo para que um elemento seja encontrado

Cálculo da altura

```
int altura(Arvore* arvore)
{
    if (vazia(arvore))
        return -1;

    int altura_esq = altura(arvore->esquerda);
    int altura_dir = altura(arvore->direita);

    if (altura_dir > altura_esq)
        return 1 + altura_dir;
    else
        return 1 + altura_esq;
}
```

Exercícios

1. Implemente uma árvore binária com as funcionalidades vistas em aula. Crie métodos para retornar
 1. A altura da árvore
 2. Se um dado elemento pertence à árvore
 3. Impressão nas 3 ordens de percurso
2. Acrescente métodos que informem se a árvore é (i) cheia, (ii) completa ou (iii) degenerada

